FunC v0.5.0
Ergonomic and Strongly Typed Design Proposal

Tomi Setsu

April 18, 2026

Document version: v4 (Draft) | **Supersedes:** v1 (initial proposal), v2 (10 Canonical Decisions locked), v3 (corpus-surfaced issues folded)

Abstract

This document proposes a new source-level design target for FunC, version `0.5.0`. It starts from the currently implemented FunC `0.4.6` surface documented in `doc/func.tex`, but reworks the language around four principles:

1. strong domain types for TVM/TOS values — typed cells, `Slice`, `Builder`, `Address`, `Coins`, and typed messages carried directly as entry-function parameters;
2. a `contract { }` block that groups state, errors, entry points, and getters into one legible declaration, with low ceremony;
3. abort-by-default semantics using `require!(cond, err)` and named exit codes, matching TVM, FunC `0.4.6`, and Solidity mental models;
4. a single `asm { }` escape hatch for direct Fift whenever the high-level surface is not enough.

FunC `0.5.0` deliberately rejects every Rust-derived feature whose complexity outweighs its benefit in the smart-contract context: no ownership, no lifetimes, no `\&self` borrows, no macros, no `async/await`, no trait objects — and **no** `?` operator. `Result<T, E>` is retained as a

niche type for genuine recovery paths, but the idiomatic failure mechanism is abort. Traits, generics, and `impl` blocks remain available for stdlib and advanced library authors, but ordinary contract authors never need to write them.

This is a design proposal, not an implementation manifest. Its role is to define a coherent, teachable next-generation FunC surface that remains faithful to TVM/TOS semantics.

Introduction

Current FunC is compact and close to TVM, but that same closeness makes it hard to learn and easy to misuse. Several recurring pain points follow directly from the `0.4.6` language shape:

- domain values are not modeled as strongly as they should be;
- boolean logic, parsing success, and protocol errors are too often represented as unstructured integers;
- mutation is hidden inside special syntax such as `~method`;
- exception codes are magic numbers rather than named constants;
- code reuse relies too heavily on low-level built-ins and ad hoc include patterns;
- entry-point message bodies must be parsed manually, one field at a time, from a raw `Slice`.

FunC `0.5.0` addresses these issues by keeping the low-level power of TVM available while moving ordinary authoring toward a safer and more legible surface. The central design choice is *one surface plus one escape hatch*: a compact high-level language that covers the vast majority of contract code, and an `asm { }` block that drops to raw Fift whenever the high level cannot express what TVM can do. This mirrors how Solidity plus its inline `assembly { }` block is used in practice on EVM, adapted for the TVM instruction set.

The intended result is not "higher level at any cost", but a better alignment between source code and the actual constraints of on-chain execution.

Contents

1 Status and Positioning

1.1 Status

This document is a **draft design proposal**. It defines the desired surface language, type system, and standard-library direction for a future FunC `0.5.0` edition. It does not claim that the current compiler already implements these features.

1.2 Relationship to FunC 0.4.6

FunC `0.4.6` is the implementation baseline. FunC `0.5.0` is the language-design target. The design aims to preserve the following invariants:

- compilation still targets TVM/Fift-style low-level code;
- TVM cells, slices, builders, continuations, and message actions remain first-class concerns;
- asynchronous cross-contract messaging remains the fundamental execution model;
- low-level escape hatches continue to exist for system code and stdlib authors.

1.3 What This Proposal Is Not

FunC `0.5.0` is *not* intended to be:

- a clone of Solidity;
- a full Rust compiler for TVM;
- a language that hides gas, bounce, serialization, or asynchronous delivery;
- a language that requires ownership and lifetime reasoning for ordinary contract code.

2 First Principles

2.1 Execution Model Comes First

The language must reflect the real execution model of TVM/TOS rather than a familiar but misleading model imported from EVM or general-purpose systems programming.

The relevant protocol facts are:

1. each contract owns isolated state;
2. cross-contract interaction happens through asynchronous messages;
3. a message may bounce, arrive later, or never arrive;
4. fees, reserve actions, and outgoing messages are explicit protocol operations;
5. serialization into cells is fundamental rather than incidental.

These facts impose direct source-language consequences:

1. contract-visible types must distinguish addresses, messages, coins, and raw cells;
2. fallibility should be part of the type system instead of being encoded as integer conventions;
3. mutation should be visible in the source rather than hidden in naming conventions;
4. side effects should be visible in signatures rather than collapsed into a single catch-all `impure` marker;
5. low-level TVM intrinsics should mostly live in the standard library, not in the parser.

2.2 Human Factors Matter

FunC `0.5.0` is optimized for the tasks that contract authors actually perform:

- define message schemas;
- parse inbound messages;
- update persistent state;
- send typed outbound messages;
- model recoverable failures clearly;
- reuse audited components without reading raw TVM stack code.

The language should therefore bias toward:

- readable declarations;
- local reasoning;
- exhaustiveness checks;
- effect visibility;
- error paths that are explicit and grep-able.

2.3 Non-Goals

To preserve approachability, FunC `0.5.0` deliberately excludes several features often associated with Rust:

- no ownership-and-lifetime borrow checker in the Rust sense;
- no lifetimes;
- no user-defined macros;
- no trait objects as a required abstraction mechanism;
- no `async/await`;
- no hidden heap allocation model.

At the same time, the language *may* include a lightweight checker for mutable-place overlap. Rejecting obviously dangerous expressions such as "mutate the same slice twice in one expression" is compatible with the goal of remaining easy to learn; it does not require authors to reason about lifetimes or ownership graphs.

3 Headline Changes with Respect to 0.4.6

Concern	**FunC 0.4.6**	**FunC 0.5.0**
Contract structure	free-standing `recv_internal`, `recv_external`, `seqno` etc. with implicit state layout	`contract { state {...} error ...; externa` `internal fn ...; get fn ...; }` as a single block
Local binding	`var x = ...` or explicit type application syntax	`let x = ...`, `let mut x = ...`, and ordinary typed bindings
Boolean values	commonly encoded as `int`	dedicated `bool` type with no implicit int-to-bool coercion
Mutating methods	`~method` syntax	ordinary method syntax with `mut self` in the signature
Error handling	`throw`, `throw_if`, `throw_unless(N, cond)` with bare numeric codes	`require!(cond, Err)` with a named error enum; the legacy numeric exit-code convention is preserved at the wrapper
Failure semantics	abort on failure (TVM-native)	**still abort on failure** — the idiomatic path uses `require!` with no `Result` wrapping. `Result<T, E>` remains available for genuine recoverable paths but is not the default

Domain types	`int/cell/slice/builder/cont` plus conventions	built-in domain types: `Cell<T>`, `Slice`, `Builder`, `Address`, `Coins`, `Bytes`, `Vec<T>`, `EitherRefOrInline<T>`, `bits512`, typed `i32..i257` / `u8..u256` scalars
Entry bodies	entry functions take a raw `slice in_msg`; every contract hand-rolls opcode load + branch + field-by-field parse	entry functions take a **typed message parameter** directly; the generated wrapper decodes the raw body into a `struct` or `enum` before calling the user body
Signature verification	hand-rolled signature+hash wiring in every wallet	`Signed<T>` stdlib wrapper: `msg: Signed<Payload>` + `msg.check(pubkey)`
Trailing-body handling	while-loop over a shared `slice`	message declarations may end with a `Slice` or `Cell` field that captures the remaining bits/refs; everything before is structurally parsed
Control over impurity	single `impure` qualifier	an effect system (`pure` / `effect(read, write, send)` / `unsafe`) that is **inferred by default**; see the Effect Visibility Policy (§10.3)

Raw TVM interop	special `asm` function bodies	`asm` function bodies + inline `asm { """ ... """ }` expression/statement blocks, both gated behind `unsafe`
Message schemas	manual op-code constants and slice plumbing	`message M(op = 0x...) { fields }` with derived codec; `enum` of messages for dispatch via `match`
Serialization	manual builder/slice plumbing for every protocol type	derived codecs via `Cell<T>`, `to_cell`, `from_cell`; `store_any` / `load_any` as stdlib helpers
Error codes	magic integers passed to `throw_unless(N, ...)`	`error A = 33, B = 34;` mini-syntax inside `contract { }` expands to a private enum with numeric discriminants preserved for ABI parity
Reuse	include-heavy and built-in-heavy	`mod` / `use` / generics / `trait` / `impl`, available but intended primarily for stdlib and library authors
Built-ins	many parser-predefined names	small core + stdlib `extern "tvm"` declarations

4 Design Goals

4.1 Primary Goals

FunC `0.5.0` has the following goals:

1. preserve TVM/TOS fidelity;

2. reduce incidental syntax complexity;
3. make domain concepts first-class in the type system;
4. make recoverable failure ordinary and typed;
5. make mutation and effects explicit;
6. make modules, traits, and generics the primary reuse mechanism;
7. keep low-level interop available for advanced code and compiler-adjacent libraries.

4.2 Compatibility Goals

The language should permit a staged migration from existing FunC contracts. Legacy code should not need to be rewritten in one step. Instead, the compiler should support an edition or compatibility mode in which old constructs are accepted while new constructs are gradually adopted.

5 Lexical Structure

5.1 Whitespace and Comments

FunC `0.5.0` adopts a more conventional comment system:

- `//` introduces a line comment;
- `/* ... */` introduces a nested block comment.

For migration purposes, the legacy FunC comment forms `;;` and `{- ... -}` may remain accepted in compatibility mode, but they are not part of the preferred `0.5.0` surface.

5.2 Identifiers

Ordinary identifiers use a conventional snake-case-oriented shape:

```
[A-Za-z_][A-Za-z0-9_]*
```

Module-qualified names use `::`:

```
std::slice::SliceReader
wallet::msg::Transfer
```

Predicate names should use `is_*` or `has_*` rather than the old `name?` convention. Fallible operations should use `try_*` or return `Result`.

5.3 Literals

The preferred literal forms are:

- integers: `0`, `17`, `0xff`, `42u32`, `-1i257`;
- booleans: `true`, `false`;
- strings: `"hello"`;
- byte strings: `b"abc"`;
- hexadecimal byte strings: `hex"deadbeef"`;
- addresses: `addr"EQC..."` or `addr"-1:abcd..."`.

The old string-suffix forms from FunC `0.4.6` may remain available behind compatibility gates, but new code should prefer explicit prefixes and typed constructors.

5.4 Keywords

The core language reserves the following keyword set:

```
mod use pub const type struct enum message trait impl fn extern
let mut match if else while loop for in break continue return
where self Self as
effect pure unsafe asm
contract state error require emit
external internal bounce get
```

`where` is reserved for future use. Compatibility mode may additionally reserve legacy FunC words such as `global`, `impure`, `inline`, and `method_id`.

`require` and `emit` are consumed by their macro forms `require!` and `emit` (see §11.4); they are not callable as plain function names. `contract`, `state`, and `error` introduce a contract block and its declarations (§6.4). `external` / `internal` / `bounce` / `get` are entry qualifiers inside a contract block.

5.5 Attributes

FunC `0.5.0` introduces an attribute syntax for edition selection and module-level metadata. Inner attributes apply to the enclosing item (here, the source file):

```
InnerAttribute ::= "#" "!" "[" AttrBody "]"
OuterAttribute ::= "#"     "[" AttrBody "]"
AttrBody       ::= identifier [ "=" (string-literal | integer-literal) ]
                 | identifier "(" [ AttrArg { "," AttrArg } ] ")"
AttrArg        ::= identifier [ "=" (string-literal | integer-literal) ]
```

The only recognised inner attribute in the edition-`0.5.0` surface is `edition`:

```
#![edition = "0.5.0"]
#![edition = "0.4-compat"]
```

Unknown attributes are reserved and reject at parse time to leave room for future extensions. Outer attributes (`#[...]`) on individual items are reserved but unused in this edition.

6 Source Structure and Modules

6.1 Compilation Unit

A source file is a sequence of items:

```
Item ::= ModuleDecl
       | UseDecl
       | ConstDecl
       | TypeAlias
```

```
    | StructDecl
    | EnumDecl
    | MessageDecl
    | TraitDecl
    | ImplDecl
    | FunctionDecl
```

6.2 Modules

Modules organize contract code and the standard library:

```
mod wallet::state;
mod wallet::msg;
use std::result::Result;
use std::message::{InternalContext, SendOptions};
```

The language keeps modules simple:

- there is no macro expansion model tied to modules;
- visibility is controlled by `pub`;
- module-qualified names use `::`;
- `use` imports names into local scope.

6.3 Public API Surface

Every item is private to its module unless marked `pub`. This matters for standard library design, contract libraries, and test-only helpers:

```
pub struct WalletState { ... }
pub enum WalletMsg { ... }
pub message Transfer(op = 0x0f8a7ea5) { ... }
pub fn helper(...) -> () effect(read) { ... }
```

6.4 The `contract { ... }` Block

The recommended top-level shape for a smart contract is a `contract { }` block that groups persistent state, error codes, entry points, getters, and contract-local helpers into one declaration:

```
contract Name {
    state { field: T, field: T, ... }

    error Code1 = N1,
          Code2 = N2,
          ...;

    message LocalMsg(op = 0x...) { ... }    // optional local schemas
    enum LocalBundle { ... }

    external fn recv(msg: SomeMessage) { ... }
    internal fn recv(msg: SomeEnum)    { ... }
    bounce   fn on_bounce(ctx: BounceContext<T>) { ... }
    get      fn name(...) -> T { ... }

    fn helper(mut state: Name, ...) { ... }  // contract-local helper
}
```

The `contract Name { }` block is a sugar that expands to:

- a `pub struct NameState { ... }` with the declared fields;
- a private `enum NameError : u16 { ... }` from the `error` declarations, with numeric discriminants preserved;
- generated `load_state`/`save_state` helpers over the persistent-data cell;
- wrapper functions around the entry points that decode the raw inbound body into the typed parameter and (on success) commit state, or (on failure) abort with the appropriate exit code;
- getters exported with their `method_id` per Canonical Decision 6;

- an ABI manifest enumerating entry effect sets, exit codes, and exported method-ids.

Authors who want finer control may still declare these items directly at module scope (without the `contract` block). The block is syntactic sugar, not a restriction.

6.5 The `error` Shorthand

Inside a `contract { }` block (or at module scope with the same syntax) authors may declare error codes using the `error` shorthand:

```
error WrongSeqno = 33,
      BadSignature = 35,
      Expired = 35,
      UnknownOp = 40;
```

This expands to a private `enum ContractError : u16` with the declared variants and numeric discriminants. The codes are used directly in `require!(cond, ErrorCode)` (see §11.4). Multiple variants may share a numeric value (as above with `35`); the compiler keeps them distinguishable at the source level while producing identical exit-code behaviour, so existing indexers that compare `exit_code == 35` still work.

The longer form `enum CustomError : u16 { ... }` remains available and is required when error variants need to carry payload or span modules (see Section 13).

7 Type System

7.1 Built-In Scalar Types

The proposed scalar types are:

```
bool
i32  i64  i128  i256  i257
u8   u16  u32  u64  u128  u256
```

`i257` remains available because TVM integers are fundamentally 257-bit signed values. For migration, `int` may remain an alias of `i257` in compatibility mode, but new code should prefer explicit widths.

7.2 Built-In Domain Types

The following types become first-class built-ins:

```
Cell<T>
Slice
Builder
Cont
Address
StdAddress
Coins
Bytes
RawCell
RawSlice
Vec<T>
EitherRefOrInline<T>
```

The role of these types is:

- `Cell<T>` represents a typed cell reference carrying an expected decoded payload type;
- `Slice` and `Builder` model the TVM read/write cell domain;
- `Address` models a full on-chain address, not an arbitrary slice;
- `Coins` models currency amounts and documents intent better than raw `u128`;
- `Bytes` represents ordinary byte strings;
- `RawCell` and `RawSlice` are low-level escape-hatch types for unchecked code;
- `Vec<T>` is a sequence type with a fixed stack and wire representation, usable at getter and ABI boundaries (§7.3);
- `EitherRefOrInline<T>` is a field-level codec that chooses inline or ref storage deterministically (§7.4).

7.3 `Vec<T>`

A `Vec<T>` is a homogeneous sequence usable wherever the built-in domain types are usable, including struct fields, function arguments, function returns, and `get fn` exports.

Stack representation. A `Vec<T>` on the TVM stack is a single TVM `Tuple` whose elements are the sequence's items in order. TVM tuples natively hold up to 255 elements; a `Vec<T>` longer than 255 is represented by a chain of tail tuples (last element of the outer tuple is itself a `Vec<T>` for the remainder). The chain is deterministic: each segment holds exactly `min(255, remaining)` elements before continuing.

Wire representation. When serialised into a cell, a `Vec<T>` writes:

1. `length:uint16` — the number of items, big-endian;
2. the items in order, each by `T`'s codec;
3. if the encoded items overflow the current cell (bit budget or ref budget), the encoder spills into a *tail ref* whose referent is a cell containing the remaining items, recursively.

Overflow is determined by the same fit predicate used in §21.4. The `uint16` prefix caps the wire length at 65 535 items; `Vec` values longer than that cannot be serialised and raise `CellError::Overflow` at the codec boundary.

Getter returns. A `get fn` returning `Vec<T>` exports the stack-tuple form directly to the caller. Indexers and wallets that consume the getter see a TVM tuple and can iterate without invoking a cell decode.

7.4 `EitherRefOrInline<T>`

This type represents a field whose TL-B form is `field:(Either T ^T)`: one bit of discriminant followed by either an inline encoding of `T` or a reference to a cell carrying `T`.

Stack representation. An `EitherRefOrInline<T>` on the TVM stack is a `T` (the inline/ref distinction is a codec-level concern, not a stack concern). The compiler carries no runtime discriminant past the codec boundary.

Wire representation. The encoder writes:

1. one discriminant bit (`0` = inline, `1` = ref);
2. if inline: the encoded `T` immediately after the bit;
3. if ref: a single cell reference whose referent is the encoded `T`.

The choice is deterministic and uses the same fit predicate as §21.4, specialised to the current field cursor: inline iff the encoded `T` fits in the remaining bits and refs of the current cell after consuming the discriminant bit. Two compliant compilers must produce identical bytes.

`EitherRefOrInline<T>` is distinct from `Option<Cell<T>>`: the former is mandatory (`T` is always present), the latter is (`0` = absent, `1` = present).

7.5 Typed Cell References

Typed cells are central to making TVM serialization tractable without hiding it:

```
pub struct WalletState {
    seqno: u32,
    owner: Address,
    plugins: Cell<PluginDict>?,
}
```

`Cell<T>` is assignable to `RawCell` when raw interop is needed, but ordinary contract code should prefer the typed form. This improves reviewability for persistent state, lazy-loaded message refs, and nested TL-B objects.

7.6 Compound Types

Compound types are:

```
(T1, T2, ..., Tn)          // tuples
[T; N]                     // fixed-size arrays
T?                         // sugar for Option<T>
Option<T>
Result<T, E>
```

Tuples remain useful because TVM naturally works with multiple stack results, but user-defined `struct` and `enum` become the preferred way to describe meaningful protocol data.

7.7 Struct Types

Structures are declared as:

```
pub struct WalletState {
    seqno: u32,
    public_key: u256,
    plugins: Dict<PluginKey, ()>,
}
```

Struct fields are named, typed, and immutable by default. Mutation happens through a mutable binding or a method with `mut self`.

7.8 Enum Types

Enumerations are algebraic data types and are central to the proposal:

```
pub enum WalletMsg {
    Transfer {
        query_id: u64,
        amount: Coins,
        to: Address,
        response_to: Option<Address>,
    },
    InstallPlugin {
        plugin: Address,
    },
    RemovePlugin {
```

```
        plugin: Address,
    },
}
```

Enums may also carry explicit discriminants when required by wire formats:

```
pub enum Op : u32 {
    Transfer = 0x0f8a7ea5,
    Burn     = 0x595f07bc,
}
```

7.9 Message Declarations

Wire-level message bodies deserve a dedicated declaration form because they are one of the dominant concepts in TVM/TOS contracts:

```
pub message Transfer(op = 0x0f8a7ea5) {
    query_id: u64,
    amount: Coins,
    to: Address,
    response_to: Address?,
}
```

A `message` declaration behaves like a named struct plus a derived message codec. In particular:

- the opcode binding is part of the declaration rather than an external convention;
- the declaration derives the relevant cell/TL-B codec;
- entry functions may take a message type directly as their parameter (§12.1); the generated wrapper decodes the inbound body into the typed struct before invoking the user body;
- an `enum` of message variants dispatches multiple opcodes through a single `match`.

Message declarations without an `op =` clause are still valid and describe a body schema without a wire-level opcode; such schemas are used as payloads inside wrappers such as `Signed<T>` or as non-TEP-owned custom formats.

Messages without opcodes as entry payloads. For external entries (notably wallet bodies), a message may be declared without an opcode:

```
pub message WalletRequest {
    global_id: i32,
    subwallet_id: u32,
    valid_until: u32,
    seqno: u32,
    actions: Slice,
}
```

This is the natural payload type for `Signed<WalletRequest>`, where the wrapper carries the signature and the message carries the signed body (see §12.1).

7.10 Trailing `Slice` / `Cell` Field

A message or struct may end with a single field of type `Slice`, `RawSlice`, `Cell`, or `RawCell`. When present, such a field **captures the remainder** of the decoder input without further structural parsing:

- a trailing `Slice` or `RawSlice` field binds to the remaining bits **and** refs of the current cell, as a native TVM slice, letting the author iterate manually or pass it to another parser;
- a trailing `Cell` or `RawCell` field binds to the next single `^Cell` reference, which the author may later decode using a more specific codec.

A `Slice`/`Cell` field in any position other than last is a compile-time error (`TailFieldNotLast`), because a non-terminal capture has no determinable boundary.

The rule lets authors model "fixed structured header followed by opaque tail" naturally, without building a custom ad-hoc decoder in every wallet contract.

7.11 Option and Result

Two standard prelude enums are defined. Their variant order is fixed by the canonical tag assignment in Section 21.1:

```
pub enum Option<T> {
    None,        // tag 0
    Some(T),     // tag 1
}

pub enum Result<T, E> {
    Ok(T),       // tag 0
    Err(E),      // tag 1
}
```

`Option<T>` is first-class. `Option<T>` expresses fields that may or may not be present (for example an optional response address on a transfer). It is used routinely; `match msg { Some(x) => ..., None => ... }` and `if let Some(x) = msg.response_to` are the ordinary idioms.

Wire form: bit-compatible with TL-B `Maybe X`. Stack form: `(tag, payload)` per Canonical Decision 1.

For readability, `T?` is accepted as surface sugar for `Option<T>`:

```
response_to: Address?
plugins: Cell<PluginDict>?
```

`Result<T, E>` is kept but deprioritised. `Result<T, E>` exists for genuine recoverable-failure paths — for example a cross-contract reply that can be `Ok` with a payload or `Err` with a typed reason, and the caller genuinely wants to branch on the two cases locally. Such paths are rare in contract code.

For the overwhelmingly common case — a value that either arrives correctly or causes the whole transaction to abort — the idiomatic form is **not** `Result`. Stdlib APIs return raw values and abort on failure, matching TVM native semantics. Authors use `require!(cond, err)` (§11.4) to assert conditions and abort with a named exit code. See Section 13 for the full error model.

FunC `0.5.0` does **not** provide a `?` operator. Recoverable-error chains that legitimately need `Result` are handled with an explicit `match` at every call site. The absence of `?` is a deliberate design choice: the `?` family of syntax encourages rescuing errors at a distance, which matches poorly with TVM's abort-or-commit transaction semantics.

7.12 Type Aliases

Type aliases remain important for readability:

```
pub type QueryId = u64;
pub type PluginKey = (i32, u256);
pub type Ton = Coins;
```

7.13 Generics

Generic functions, structs, enums, and traits use angle-bracket syntax:

```
pub struct Dict<K, V> { ... }

pub fn first<T>(items: (T, T)) -> T { ... }
```

Unlike Rust, the design does not introduce lifetime parameters or ownership bounds. Generic programming is deliberately restricted to type parameters and trait constraints.

8 Traits and Implementations

8.1 Trait Declarations

Traits provide interface-oriented reuse:

```
pub trait TlbCodec {
    fn store(self, mut b: Builder) -> Result<Builder, CellError> pure;
    fn load(mut s: Slice) -> Result<(Self, Slice), ParseError> pure;
}
```

Inside a `trait` or `impl` body, `Self` refers to the concrete type that implements (or is being defined by) the surrounding block. Outside these blocks `Self` is not in scope. A trait method whose signature does not name `self` is a **static associated function** and is called with explicit type syntax, e.g. `WalletMsg::load(cs)`. Methods with a `self` or `mut self` receiver are dispatched through ordinary method-call syntax `x.method(...)`. The proposal does *not* include borrowed receivers; there are no `\&self` or `\&mut self` forms.

8.2 Implementations

Implementations attach methods to types:

```
impl WalletState {
    pub fn increment_seqno(mut self) -> Self pure {
        self.seqno += 1;
        self
    }
}

impl TlbCodec for WalletMsg {
    fn store(self, mut b: Builder) -> Result<Builder, CellError> pure { ... }
    fn load(mut s: Slice) -> Result<(Self, Slice), ParseError> pure { ... }
}
```

8.3 Why Traits Instead of Parser Magic

One design goal of `0.5.0` is to move functionality out of the parser and into audited, inspectable library code. Traits support that directly:

- codecs can be trait-based;
- message sending can depend on a trait bound rather than raw cells;
- collection and builder helpers can live in the standard library;
- test doubles and mock implementations become natural.

This does *not* mean the compiler should be blind to protocol structure. A small amount of protocol-aware lowering remains justified for things such as entrypoint contexts, message opcodes, and automatic inline-vs-ref message-body placement. The goal is to eliminate accidental parser magic, not to forbid every useful built-in lowering.

9 Bindings, Mutability, and Patterns

9.1 Immutable by Default

All local bindings are immutable unless declared with `mut`:

```
let seqno: u32 = state.seqno;
let mut cs: Slice = body;
```

This replaces the old mixture of `var`, explicit type application, and hidden mutation through `~method` calls.

9.2 Assignments

Only mutable places may be assigned to:

```
let mut state = load_state()?;
state.seqno = state.seqno + 1;
```

9.3 Pattern-Oriented Control Flow

Patterns are not limited to `match`. The language should also support `if let` and `while let` because optional values and typed message variants are ubiquitous in contract code:

```
if let Some(response_to) = msg.response_to {
    send(response_to, Ack { query_id: msg.query_id }, opts)?;
}

while let Some(item) = queue.pop_front() {
    process(item)?;
}
```

9.4 Destructuring

Patterns work in `let` bindings and in `match`:

```
let (amount, response_to) = (msg.amount, msg.response_to);
let WalletMsg::Transfer { amount, to, .. } = msg;
```

Pattern forms include:

- wildcard `_`;
- name bindings;

- tuple patterns;
- struct patterns;
- enum variant patterns.

9.5 Mutating Methods

Methods may declare `mut self`:

```
impl Slice {
    pub fn load_uint(mut self, bits: u16) -> Result<u256, ParseError> pure { ... }
}
```

At the call site, this looks ordinary:

```
let mut cs = body;
let op = cs.load_uint(32)?;
```

This replaces the special `cs~load_uint(32)` syntax. The source clearly states mutability in the binding and in the method signature, rather than hiding it in punctuation.

Mutation on the abort path. When a `mut self` method aborts (via `require!`, an overflowing cell operation, or any other `THROW`), the entire transaction rolls back: persistent state is not committed, the action list is discarded, and the caller's mutable place is irrelevant because execution halts. Authors do not reason about partial state updates across an abort boundary.

9.6 Mutable Arguments and Lightweight Safety Checks

Methods with `mut self` are the idiomatic surface for builder/slice-like APIs. Free functions — and in particular helper functions that update a contract's persistent state — accept mutable places explicitly through a `mut` parameter qualifier:

```
pub fn load_var_uint(mut s: Slice, bits: u16) -> u256 pure { ... }

let mut cs = body;
let n = load_var_uint(mut cs, 5);   // aborts on malformed input
```

Helper functions that mutate persistent state. This is the canonical pattern for splitting an entry body into smaller helpers that each mutate the protocol state. There is no `&self` or `&mut self` borrow form in FunC `0.5.0`; state passes through `mut` arguments by value-thread:

```
fn on_transfer(mut state: WalletState, sender: Address, req: Transfer) { ... }

internal fn recv(msg: WalletMsg) {
    let mut state = load_state();
    match msg {
        WalletMsg::Transfer(req) => on_transfer(mut state, ctx.sender, req),
        WalletMsg::Burn(req)     => on_burn(mut state, ctx.sender, req),
        ...
    }
    save_state(state);
}
```

The lowering passes the receiver by value; the helper returns the updated receiver and its own return value (if any), and the caller's binding is rebound to the updated receiver. An abort along the helper path unwinds the whole transaction, so no per-place rollback logic is required.

Overlap checks. This proposal adopts three lightweight rules:

- only mutable lvalues may be passed as `mut` arguments;
- the same mutable place may not be passed twice in one full expression;
- parent and child places may not be mutated simultaneously in one full expression.

These rules are intentionally narrow. They prevent obvious aliasing hazards without introducing ownership, region inference, or lifetime syntax.

10 Functions and Effect System

10.1 Function Syntax

The general form is:

```
[pub] fn name(arg1: T1, mut arg2: T2, ...) -> R [EffectQualifier] Block
EffectQualifier ::= "pure" | "unsafe" | "effect" "(" EffectList ")"
```

Examples:

```
pub fn min(x: i257, y: i257) -> i257 pure { ... }
pub fn load_state() -> WalletState effect(read) { ... }
pub fn load_uint(mut s: Slice, bits: u16) -> u256 pure { ... }
pub fn send_transfer(...) -> () effect(send) { ... }
```

10.2 Effect Vocabulary

The effect vocabulary is:

- `read`: may inspect persistent state, inbound context, config, time, balance, or other chain environment values;
- `write`: may persist updated state or otherwise mutate contract-owned storage;
- `send`: may emit action-phase effects such as outbound messages, reserve operations, or code changes.

An effect set is cumulative: `effect(write, send)` may both write state and send messages. A `pure` function has the empty effect set; it may only call other `pure` functions. Effects propagate transitively: if callee's effect set is E, caller's effect set must be a superset of E.

`unsafe` is *not* a member of the effect set. It is a separate top-level qualifier that grants permission to use raw `asm` blocks and `extern "tvm"` intrinsics whose safety cannot be proved from the surface type rules alone. A function that contains an `asm` block (§10.7) must be declared `unsafe`; callers of `unsafe` functions must themselves be `unsafe` or wrap the call in a way that re-establishes the invariants (the usual pattern is a thin safe wrapper around an `unsafe` primitive).

`mut self` and `mut` arguments are *not* effects; they express local state threading and are compatible with any effect set including `pure`.

10.3 Effect Visibility Policy

Effects are **inferred by default**. Source code does not need to declare them in most positions. The spec fixes the visibility policy as follows:

Function kind	Effect visibility	Why
Local `fn` inside a `contract { }` block or inside a module	**inferred**, not written in source	noise for the common case; compiler still computes and propagates
`get fn` inside a `contract { }` block	**inferred with ceiling** `effect(read)` — writing or sending from a getter is a compile error	the `get` keyword itself declares the effect ceiling; no need to repeat
`external fn` / `internal fn` / `bounce fn` inside a `contract { }` block	**inferred; upper bound** `effect(read, write, send)`	the entry keyword declares the ceiling; most entries hit the full set anyway
`pub fn` in any module (library-visible)	**explicit**; the declared effect set is part of the stable ABI	cross-module callers need to see the effect set without inference; stdlib and library authors must not break the declared set silently
`unsafe` functions	**always written in source** as a top-level qualifier; not inferable	`unsafe` is a security boundary; auditors must see it and it propagates transitively

Opt-in restriction. An author may write an `effect(...)` annotation on any function to *restrict* its inferred set. For example a critical function may declare `effect(read)` to have the compiler enforce that no `write` or `send` sneaks in via a later refactor. This is defensive annotation, not mandatory.

ABI manifest. The compiler emits an ABI manifest alongside every contract that enumerates, for each entry and getter, the inferred effect set, exit code catalogue, exported `method_id` values, and declared messages. Auditors, indexers, wallets, and block explorers consume this manifest as the

source of truth for effect information rather than relying on source-level annotations.

Why Not Keep `impure`. The old `impure` marker is too coarse to distinguish a getter that reads block context from a function that mutates storage and sends three messages. The inferred effect set preserves the fine-grained distinction without forcing authors to write it.

10.4 Entrypoints

Entry qualifiers live inside a `contract { }` block. The entry function takes its inbound body directly as a typed parameter:

```
contract Wallet {
    state { ... }
    error ... ;

    external fn recv(msg: Signed<Payload>) { ... }
    internal fn recv(msg: WalletMsg)        { ... }
    bounce   fn on_bounce(ctx: BounceContext<Transfer>) { ... }
    get      fn seqno() -> u32 { state.seqno }
}
```

The compiler generates a wrapper per entry that decodes the raw inbound body into the declared parameter type using the derived codec (see Section 21.1), then calls the user body. Success commits state; any abort along the user-body path rolls back the whole transaction.

Inside a user body the names `state`, `ctx.sender`, `ctx.value`, and `ctx.forward_fee` are in scope when applicable (`ctx` is supplied only where the entry receives a context type, not a plain typed body). `state` is a mutable reference to the contract's persistent state, rebound to the updated value on normal return.

The context types that can appear as an entry parameter are:

```
Signed<T>                // external wallet-like pattern (stdlib; §9.2)
WalletMsg, Transfer, ... // any user-defined typed message
BounceContext<T>         // for entry bounce
SignedMessage<T>         // alias for Signed<T>
```

A plain `Slice` parameter is also legal and falls back to raw body processing; it is the low-ceremony escape when the typed-entry pattern does not fit.

10.5 Entry Dispatch Matrix

Unlike FunC `0.4.6` which uses a single `recv_internal` and expects the contract to branch on the `bounced` header bit, `0.5.0` splits delivery into three independent entry families:

- fresh internal messages are delivered to `internal fn`;
- bounced messages are delivered to `bounce fn`;
- external messages are delivered to `external fn`.

If a contract defines `internal fn` but no `bounce fn`, an inbound bounced message is silently dropped after consuming only the minimal gas needed by the low-level wrapper — no user-written handler is invoked. Symmetric rules apply to undefined `internal fn` (fresh internal silently dropped) and undefined `external fn` (external message rejected at the wrapper with exit code 11 = invalid operation). Contracts requiring the legacy single-entry pattern use a `Slice` parameter and manually re-dispatch.

10.6 Entry Decoding Semantics

Typed body decoding happens in the generated wrapper, not in user code. The canonical semantics for decode failure are:

- `internal fn recv(msg: T)`: if the inbound body cannot be decoded as `T`, the wrapper exits successfully without invoking the user body. No state is written, no messages are sent. This preserves the TOS convention that unrecognized ops do not cause bounce-refund gas drain. For a generic fallback that inspects any body, use `Slice` as the parameter type.
- `external fn recv(msg: T)`: if the body cannot be decoded as `T`, the wrapper aborts with exit code 9 (cell underflow) — the external message is rejected. The external-message gas limit (typically 10 000 gas

before `accept_message`) applies to the decode cost; authors using a `Slice` parameter take full manual control and are responsible for calling `accept_message` at the right point. For the common typed case, the fixed header portion of a wallet-style `Signed<Payload>` is cheap enough to decode under the quota.

- `bounce fn on_bounce(ctx: BounceContext<T>)`: decoding uses the bounced-message body convention — at most 256 bits after the `0xFFFFFFFF` header, and no refs. See the bounced body width rule below.

Bounced body width rule. For any `bounce fn ...(ctx: BounceContext<T>)` with `T` not equal to `Slice` / `RawSlice`, the compiler must statically verify

$$\texttt{max\_encoded\_bits}(T) \;\leq\; 256$$

where `max_encoded_bits` is the worst-case bit width of `T`'s derived codec, not counting refs. The `0xFFFFFFFF` bounce header prefix occupies 32 bits and is added by the wrapper. Ref counts are also verified: `T` must encode as purely inline bits (no refs) because bounced bodies are truncated, not referenced.

If `max_encoded_bits(T) > 256` or `T` requires any refs, the compile error is `BounceBodyTooWide`, and the author must either narrow `T`'s schema or fall back to `BounceContext<Slice>` and parse manually.

10.7 External Intrinsics and the `asm` Escape

Raw TVM primitives are declared as `extern "tvm"` functions in the standard library rather than being magic parser names:

```
extern "tvm" fn send_raw_message(msg: RawCell, mode: u8) -> () unsafe;
extern "tvm" fn accept_message() -> () unsafe;
```

Additionally, arbitrary Fift snippets may appear in three places inside `unsafe` functions:

```
// (1) function-level asm body
fn hash(s: Slice) -> u256 unsafe asm "HASHSU";

// (2) multi-line asm body
fn crc32(data: Bytes) -> u32 unsafe asm {
```

```
    """
    NEWC
    STSLICE
    ENDC
    HASHCU
    """
}

// (3) inline asm statement / expression inside a function
fn combine(x: u256, y: u256) -> u256 {
    // ... ordinary FunC above ...
    let combined = asm(x, y) -> (u256) {
        """
        PAIR
        HASHCU
        """
    };
    combined
}
```

Rules for `asm` blocks.

- Any function that contains an `asm` block (function-level or inline) must carry the `unsafe` qualifier.
- Inline `asm(in\_vars) -> (out\_tys) { """..." "" }` must declare its input-variable list and output-type tuple. The compiler does not inspect the Fift; it trusts the declared stack contract.
- Formal parameters and `let`-bound names in the enclosing function are visible inside `asm` blocks.
- `asm` contents use the 0.4.6 Fift conventions documented in `doc/func.tex` §5.4 (AsmBody).

The combination of a compact high-level surface and an `asm` escape hatch is the core design choice of FunC `0.5.0`: the high-level covers the vast majority of contract code; `asm` covers everything TVM can do that the high-level cannot express.

11 Expressions and Statements

11.1 Core Statements

The language supports:

```
let                         // immutable binding
let mut                     // mutable binding
if / else                   // conditional
if let                      // conditional pattern bind
match                       // exhaustive multiway
while, while let, loop      // loops
for                         // iterator-driven loop
break, continue, return     // control flow
require!(cond, err)         // abort if !cond
emit Event { ... }          // emit an event message
```

11.2 Match Expressions

Pattern matching is exhaustive by default:

```
match msg {
    WalletMsg::Transfer { query_id, amount, to, response_to } => { ... }
    WalletMsg::InstallPlugin { plugin } => { ... }
    WalletMsg::RemovePlugin  { plugin } => { ... }
}
```

A non-exhaustive `match` is a compile-time error; the author must either add the missing variant or an explicit wildcard arm `_ => ...`. This is a major ergonomics improvement over manually parsing an opcode into an `int` and maintaining a parallel maze of `if` branches.

11.3 Flow-Sensitive Narrowing

Pattern matching and optional-value tests refine types in a flow-sensitive way:

```
if let Some(reply_to) = msg.response_to {
    send(reply_to, Receipt { query_id: msg.query_id }, opts);
}
```

Inside the `if let` body, `reply_to` is an `Address` rather than `Address?`. Similar narrowing happens for `match` arms over `Option` and enum / message variants.

11.4 `require!` and `emit`

FunC `0.5.0` provides two compiler-recognised forms that look like function calls but desugar at compile time.

`require!(cond, err).` Asserts `cond`; if false, aborts the transaction with the numeric exit code associated with `err`. `err` is typically a variant of the contract-local `error` enum (§6.5) or of an explicitly declared `enum E : u16`.

```
require!(seqno == state.seqno,        WrongSeqno);
require!(valid_until > now(),         Expired);
require!(amount > Coins::zero(),      ZeroAmount);
```

Desugar:

```
require!(COND, ERR)
  ==>  if !(COND) { throw(ERR as u16) }
```

`emit Event { ... }.` Emits an event message onto a predefined log sink. `Event` must be a user-declared `message` (with or without an opcode); the sink address is a compile-time constant configured in the stdlib (typically `0:FFF...FFF`, the TON log-sink convention). The payload is derived-codec-encoded and sent with mode 32 (carry remaining value + destroy-if-zero).

```
message TransferLog(op = 0xTL) {
    from:   Address,
    to:     Address,
    amount: Coins,
}

emit TransferLog { from: state.owner, to: req.to, amount: req.amount };
```

Desugar:

```
emit Event { FIELDS }
  ==>  send(log_sink_address, Event { FIELDS }, EVENT_DEFAULT_OPTS);
```

where `EVENT_DEFAULT_OPTS` is `SendOptions { value: 0, bounce: NoBounce, body_layout`

11.5 No `?` Operator

FunC `0.5.0` does not include a `?` operator. Stdlib functions either return a raw value (aborting on failure) or return an `Option<T>` / `Result<T, E>` that the caller destructures explicitly with `match` or `if let`. The design rationale is two-fold:

1. The `?` operator encourages rescuing errors at a distance, which matches poorly with TVM's abort-or-commit transaction semantics. When a smart-contract operation fails, the overwhelmingly correct action is to roll the whole transaction back, not to continue execution with a recovered error;

2. abort-by-default aligns FunC `0.5.0` with TVM native instruction semantics (`LDU` / `LDI` etc. throw on failure), with FunC `0.4.6` `~method` idioms, and with Solidity's `require(cond, "msg")`. The conceptual model is shared across three widely understood systems.

Contracts that genuinely need recoverable error paths use `Result<T, E>` and a `match` arm. Such paths are rare; see §13.

11.6 Loop Design

The preferred loop forms are:

```
while cond { ... }
loop { ... }
for item in items { ... }
```

Legacy `repeat` and `do ... until` may remain in compatibility mode, but the core language should converge on a smaller, more conventional control surface.

Iterator protocol for `for`. `for item in items` desugars to a `while let Some(item) = iter.` loop where `iter` is derived from `items` by the `Iter` trait:

```
pub trait Iter {
    type Item;
    fn next(mut self) -> Option<Self::Item> pure;
```

```
}

pub trait IntoIter {
    type Item;
    type IntoIterTy: Iter<Item = Self::Item>;
    fn into_iter(self) -> Self::IntoIterTy pure;
}
```

The desugaring rule is:

```
for <pattern> in <expr> { <body> }

    ==>

let mut __it = <expr>.into_iter();
while let Some(<pattern>) = __it.next() { <body> }
```

In edition `0.5.0` the compiler must accept `for` only over iterables whose type implements `IntoIter`. The standard library provides initial implementations for ranges, fixed arrays, tuples of uniform element type, and dictionary iteration; additional implementations may be added by stdlib or user code. Trait resolution is static (monomorphized), matching the lowering rule in `doc/func_v0.5.0-lowering.tex` §4.10.

12 Messages, Serialization, and TVM-Specific APIs

12.1 Entry Parameter Types

Entry functions take their inbound body directly as a typed parameter. Three shapes are supported:

1. A user-declared `message` type. For internal messages with an opcode, use a `message`:

```
message Transfer(op = 0x0f8a7ea5) {
    query_id: u64,
```

```
    amount: Coins,
    to: Address,
}

contract Jetton {
    internal fn on_transfer(msg: Transfer) { ... }
}
```

The wrapper consumes 32 bits of opcode, verifies it equals `0x0f8a7ea5`, decodes the fields, and calls the user body. Opcode mismatch → silent drop.

2. An `enum` of messages. For contracts handling multiple opcodes, group them in an enum:

```
enum JettonMsg {
    Transfer(Transfer),
    InternalTransfer(InternalTransfer),
    Burn(Burn),
}

contract Jetton {
    internal fn recv(msg: JettonMsg) {
        match msg {
            JettonMsg::Transfer(t)         => on_transfer(mut state, t),
            JettonMsg::InternalTransfer(i) => on_internal_transfer(mut state, i),
            JettonMsg::Burn(b)             => on_burn(mut state, b),
        }
    }
}
```

The wrapper peeks the first 32 bits and dispatches to the matching variant's decoder. No match → silent drop.

3. A `Signed<T>` wrapper for external wallet-style bodies. External wallet bodies have a standard shape — 512-bit signature followed by a signed payload — and are represented by the stdlib wrapper:

```
// std::message
```

```
message Signed<T> {
    signature: bits512,
    body: T,
}

impl<T: CellEncode> Signed<T> {
    pub fn check(self, pubkey: u256) -> bool pure;
}
```

`Signed<T>::check(pubkey)` hashes `self.body` and verifies the signature against the supplied public key. Authors use it as:

```
message Payload {
    global_id: i32,
    subwallet_id: u32,
    valid_until: u32,
    seqno: u32,
    actions: Slice,   // trailing
}

external fn recv(msg: Signed<Payload>) {
    let p = msg.body;
    require!(p.valid_until > now(),                Expired);
    require!(p.seqno == state.seqno,               WrongSeqno);
    // ...
    require!(msg.check(state.pubkey),              BadSignature);
    accept_message();
    // ... act on p.actions ...
}
```

12.2 BounceContext<T>

For bounce handlers the entry receives a context wrapper because bounced messages carry a `0xFFFFFFFF` header and protocol metadata beyond the body itself:

```
struct BounceContext<T = Slice> {
    sender: Address,
```

```
    value:  Coins,
    body:   T,
}

contract Jetton {
    bounce fn on_bounce(ctx: BounceContext<InternalTransfer>) {
        let mut state = load_state();
        state.balance += ctx.body.amount;   // refund
        save_state(state);
    }
}
```

The `T` of a `BounceContext<T>` must satisfy the bounced body width rule (§10.6): at most 256 encoded bits and no refs.

12.3 Typed Cell APIs

Serialization is trait-driven with convenience methods:

```
pub trait CellEncode {
    fn encode(self) -> RawCell pure;
    fn to_cell(self) -> Cell<Self> pure { Cell::from_raw(self.encode()) }
}

impl<T: CellDecode> Cell<T> {
    pub fn load(self) -> T pure;           // aborts on decode failure
    pub fn begin_parse(self) -> Slice pure;
}
```

Normal cases do not return `Result`; malformed cells abort the transaction. This matches the TVM opcodes `CTOS` (fails if not cell), `LDU` (fails if underflow), and the design of every real contract under 0.4.6.

12.4 Codec Traits

```
pub trait CellEncode           { fn encode(self)   -> RawCell pure; }
pub trait CellDecode: Sized    { fn decode(c: RawCell) -> Self pure; }
```

Derived automatically for every `struct`, `enum`, and `message` declaration. Authors rarely write `impl CellEncode` by hand; the stdlib derives it.

For wire protocols that rely on TL-B, a narrower `TlbCodec` trait specialises the same idea.

12.5 Slice and Builder APIs

Low-level APIs abort on failure:

```
impl Slice {
    pub fn load_uint(mut self, bits: u16) -> u256 pure;
    pub fn load_int(mut self, bits: u16)  -> i257 pure;
    pub fn load_ref(mut self)              -> RawCell pure;
    pub fn load_addr(mut self)             -> Address pure;
    pub fn remaining_bits(self) -> u16 pure;
    pub fn remaining_refs(self) -> u8  pure;
}

impl Builder {
    pub fn store_uint(mut self, value: u256, bits: u16) -> Self pure;
    pub fn store_int (mut self, value: i257, bits: u16) -> Self pure;
    pub fn store_ref (mut self, cell: RawCell)          -> Self pure;
    pub fn end_cell(self) -> RawCell pure;
}
```

Parsing and building remain explicit operations. The language does not try to make TVM cells disappear; it makes them typed and tractable.

12.6 Sending Messages

High-level message sending takes typed bodies and an options record:

```
pub enum BounceMode {
    NoBounce,
    Body256,
    Rich,
}
```

```
pub enum BodyLayout {
    Auto,
    Inline,
    ByRef,
}

pub struct SendOptions {
    value: Coins,
    bounce: BounceMode,
    body_layout: BodyLayout,
    mode: SendMode,
}

pub fn send<T: CellEncode>(
    dst: Address,
    body: T,
    opts: SendOptions,
) -> () effect(send);
```

This API expresses a real protocol operation more directly than ad hoc raw-cell construction plus `SENDRAWMSG`. `send` aborts on underlying failure (e.g. cell overflow); the result type is `()`.

When `body_layout` is `Auto`, the compiler *must* choose between inline and ref according to the canonical fit predicate in Section 21.4. The predicate is a deterministic function of `(env_bits, env_refs, body_bits, body_refs)` and is stable across toolchains.

When `body` is the unit value `()`, the generated envelope encodes "no body": the TL-B `Maybe` bit for `body:(Maybe \^{}Cell)` is set to `0`.

13 Error Model

13.1 Abort by Default

FunC `0.5.0` uses abort-by-default semantics. When a validation condition fails or an operation cannot complete (cell underflow, cell overflow, integer overflow, unauthorized sender, etc.) the transaction aborts: persistent state is not committed, the action list is discarded, no outbound messages are sent, and the contract returns a numeric exit code to the TVM level. Indexers,

wallets, and block explorers observe the exit code and treat the transaction as failed.

This matches three pre-existing models that FunC authors already know:

- TVM native opcodes (`LDU`, `LDI`, `CTOS`, ...) throw on failure;
- FunC `0.4.6` `throw_unless(N, cond)` aborts on false;
- Solidity `require(cond, "msg")` reverts on false.

The primary mechanism to assert in source is `require!(cond, err)` (§11.4). A false `cond` aborts with the numeric exit code derived from `err`.

13.2 Named Error Codes

Contracts declare their error codes inline using the `error` shorthand inside the `contract { }` block (§6.5):

```
contract Wallet {
    error WrongSeqno     = 33,
          WrongSubwallet = 34,
          BadSignature   = 35,
          Expired        = 35,
          WrongGlobal    = 36;
    ...
}
```

The shorthand expands to `pub enum WalletError : u16 { ... }` with the declared numeric discriminants. `require!(cond, WalletError::WrongSeqno)` (shortened to `require!(cond, WrongSeqno)` inside the block) aborts with exit code 33, matching `throw_unless(33, cond)` in a 0.4.6 contract byte-for-byte.

Two variants may share the same numeric value (as `Expired = 35` and `BadSignature = 35` above). The source treats them as distinct for readability and grepping; the wire code is identical.

Reserved ranges.

- `0`: normal success. Assigning `0` to an error variant is a compile-time error.
- `1` – `10`: TVM-internal exit codes (e.g. 8 = range check failed, 9 = cell underflow, 10 = dict error). Authors may re-use these codes with care when their semantics match.
- `11` – `31`: TOS-reserved for protocol-level failures (unknown operation, insufficient fee, etc.).
- `32` – `65535`: user range. Established TON wallet and Jetton conventions occupy `32` – `400`; contracts may use anything not already reserved in their domain.

13.3 The Rare `Result<T, E>` Use Case

Some protocol patterns genuinely need recoverable error paths: a cross-contract query returns an `Ok(value)` or `Err(reason)` that the caller inspects and chooses between two follow-up actions. For these rare cases, `Result<T, E>` is available as an ordinary enum:

```
enum LookupResult {
    Ok(Entry),
    Err(LookupError),
}

fn maybe_lookup(k: Key) -> LookupResult effect(read) { ... }

internal fn recv(ctx: InternalContext<LookupRequest>) {
    match maybe_lookup(ctx.body.key) {
        LookupResult::Ok(e) => send(ctx.sender, FoundReply { data: e }, ...),
        LookupResult::Err(_) => send(ctx.sender, NotFoundReply { }, ...),
    }
}
```

This is *not* the default idiom. Authors who find themselves wanting `Result` for validation-style logic should use `require!` instead — it is shorter, cheaper, and matches the abort-by-default contract model.

13.4 Bridging Legacy Exit Codes

Any FunC `0.5.0` contract that interoperates with existing indexers, wallets, or explorer tools assigns the legacy numeric codes (33, 34, 35, ...) directly in its `error` shorthand. The wire behaviour is identical to 0.4.6, and legacy tooling continues to function without change.

14 Standard Library Direction

14.1 Small Core, Rich Stdlib

The language core should stay small. Most functionality that is currently special or parser-defined should instead move to the standard library:

- arithmetic helper functions;
- slice and builder utilities;
- dictionary abstractions;
- message and context types;
- codec traits and implementations;
- contract-state helpers;
- cryptographic intrinsics surfaced as `extern "tvm"` functions.

Expected stdlib conveniences include:

- `create_message` and `send` wrappers over raw action lists;
- typed state load/save helpers built on `Cell<T>`;
- `load_any` / `store_any` helpers for routine TL-B work;
- message-body declarations with derived codecs.

14.2 Traits as the Unit of Reuse

Traits provide the right level of abstraction for:

- codecs;
- formatting and debugging;
- message sending;
- dictionary key/value constraints;
- reusable protocol behaviors.

This gives TOS a realistic path toward an audited contract standard library without forcing all composition through includes and naming conventions.

15 What Remains Explicit

The proposal intentionally keeps several low-level concerns visible:

- serialization into cells;
- bounce handling;
- coins attached to outgoing messages;
- effectful operations such as `send` and reserve actions;
- unsafe raw TVM interop where needed.

This is a language for TVM/TOS, not an attempt to hide the platform.

16 Compatibility and Migration

16.1 Edition Model

The recommended migration path is edition-based. A source file may opt into the new syntax and semantics explicitly:

```
#![edition = "0.5.0"]
```

Legacy files may continue to compile under a compatibility edition:

```
#![edition = "0.4-compat"]
```

16.2 Compatibility Surface

The compatibility edition may continue to accept:

- legacy type names such as `int` and `cell`;
- legacy comments;
- legacy `global`, `const`, `asm`, `impure`, `inline`, `method_id` forms;
- legacy `~method` mutation syntax;
- legacy exception-style helpers.

16.3 Illustrative Source Migration

```
// 0.4.6 style
var cs = in_msg;
var signature = in_msg~load_bits(512);
throw_unless(33, msg_seqno == stored_seqno);

// 0.5.0 style
let mut cs = in_msg;
let signature = cs.load_bits(512)?;
if msg_seqno != stored_seqno {
    return Err(WalletError::WrongSeqno);
}
```

16.4 Migration Principle

The important migration principle is semantic, not cosmetic:

- move from raw integers toward explicit types;

- move from hidden mutation toward `mut`;
- move from integer-status parsing toward `Result`;
- move from raw op-code dispatch toward `enum + match`;
- move from parser magic toward stdlib declarations, while retaining a small amount of protocol-aware lowering where it clearly improves source-level clarity.

17 Worked Comparison: One Wallet Contract in 0.4.6 and 0.5.0

The following comparison shows the same piece of contract logic in both language styles:

1. load wallet state;
2. parse an inbound transfer request;
3. validate the transfer sequence number;
4. send native coins to the destination;
5. increment and persist the sequence number.

The examples are intentionally compact. They are not meant to be full production wallets; they are meant to expose the source-language shape of the same task.

17.1 FunC 0.4.6 Style

In current-style FunC, the code is dominated by raw slice parsing, implicit mutation through `~method` syntax, integer-coded failures, and manual message construction:

```
;; FunC 0.4.6 style

(int, slice) load_data() inline {
```

```
  var ds = get_data().begin_parse();
  return (ds~load_uint(32), ds~load_msg_addr());
}

() save_data(int seqno, slice owner) impure inline {
  set_data(
    begin_cell()
      .store_uint(seqno, 32)
      .store_slice(owner)
      .end_cell()
  );
}

() recv_internal(slice in_msg_body) impure {
  var (stored_seqno, owner) = load_data();
  var cs = in_msg_body;

  int op = cs~load_uint(32);
  throw_unless(40, op == 0x0f8a7ea5);

  int msg_seqno = cs~load_uint(32);
  slice to = cs~load_msg_addr();
  int amount = cs~load_tomis();

  throw_unless(33, msg_seqno == stored_seqno);
  throw_unless(34, amount > 0);

  send_raw_message(
    begin_cell()
      .store_uint(0x18, 6)
      .store_slice(to)
      .store_tomis(amount)
      .store_uint(0, 1 + 4 + 4 + 64 + 32 + 1 + 1)
      .end_cell(),
    1
  );

  save_data(stored_seqno + 1, owner);
```

```
}
```

17.2 FunC 0.5.0 Style

In the proposed `0.5.0` surface, the same logic is expressed through a `contract` block, a typed message, named error codes, and `require!` for validation:

```
#![edition = "0.5.0"]

contract Wallet {
    state { seqno: u32, owner: Address }

    error WrongSeqno = 33,
          ZeroAmount = 34,
          UnknownOp  = 40;

    message Transfer(op = 0x0f8a7ea5) {
        seqno:  u32,
        to:     Address,
        amount: Coins,
    }

    internal fn on_transfer(msg: Transfer) {
        require!(msg.seqno == state.seqno,  WrongSeqno);
        require!(msg.amount > Coins::zero(), ZeroAmount);

        send(
            msg.to,
            (),
            SendOptions {
                value:       msg.amount,
                bounce:      BounceMode::Body256,
                body_layout: BodyLayout::Auto,
                mode:        SendMode::PayFeesSeparately,
            },
        );

        state.seqno += 1;
```

```
    }
}
```

17.3 What the Comparison Shows

The comparison illustrates the intended source-level improvements:

- **Contract block.** The `contract Wallet \{ state, error, message, internal fn \}` shape groups persistent state, error codes, message schemas, and entry points into one declaration, eliminating imports, explicit `load_data` / `save_data` helpers, and boilerplate per-entry setup.
- **Typed message declaration instead of raw op-code dispatch.** In `0.4.6` the contract manually loads `op`, then loads each field from a `slice`. In `0.5.0` the `Transfer` message binds its opcode in the declaration and the entry receives a typed body directly. Opcode mismatch is handled by the wrapper (silent drop), not by user code.
- **Explicit domain types.** `Address` and `Coins` replace raw `slice` and `int` conventions, making the meaning of each field visible at the type level.
- **Explicit mutation.** In `0.4.6` mutation is hidden inside `cs~load_uint(...)` and similar forms. In `0.5.0` persistent state is `state` and mutates in place; parameter-level mutation uses `let mut` and `mut` arguments.
- **Named error codes, legacy wire behaviour preserved.** `throw_unless(33, ...)` and `throw_unless(34, ...)` become `require!(cond, WrongSeqno)` and `require!(cond, ZeroAmount)`. The numeric exit codes (33, 34) are preserved via the explicit discriminants, so existing indexers continue to work.
- **High-level send API instead of manual action-cell assembly.** The `0.4.6` version builds an outbound message cell explicitly. The `0.5.0` version exposes the protocol operation as `send(...)`, with value, bounce mode, body layout, and mode still explicit in the options record.
- **No `?` operator, no `Result` clutter.** Failures abort the transaction. The source code reads as a linear sequence of checks and actions, matching the mental model of `require(cond)` from Solidity and `throw_unless(N, cond)` from FunC `0.4.6`.

17.4 Why This Example Matters

This example is deliberately small, but the same pattern scales to larger contracts. See the Canonical Corpus (`doc/func_v0.5.0-corpus.md`) for worked rewrites of wallet-v4, highload-v2, and a synthetic Jetton wallet, each under 50 lines in the new surface.

18 Core Grammar Summary

The grammar below is intentionally schematic. It describes the shape of the language, not every precedence rule or parsing ambiguity.

```
SourceFile ::= { InnerAttribute } { Item }

Item ::= { OuterAttribute }
         ( ModuleDecl
         | UseDecl
         | ConstDecl
         | TypeAlias
         | StructDecl
         | EnumDecl
         | MessageDecl
         | TraitDecl
         | ImplDecl
         | FunctionDecl )

ModuleDecl ::= "mod" Path ";"
UseDecl    ::= "use" Path [ "as" identifier ] ";"
ConstDecl  ::= [ "pub" ] "const" identifier ":" Type "=" Expr ";"
TypeAlias  ::= [ "pub" ] "type" identifier [ GenericParams ] "=" Type ";"

StructDecl ::= [ "pub" ] "struct" identifier [ GenericParams ] "{" Fields "}"
EnumDecl   ::= [ "pub" ] "enum" identifier [ GenericParams ] [ ":" Type ] "{"
               Variants "}"
MessageDecl ::= [ "pub" ] "message" identifier [ GenericParams ]
                [ "(" "op" "=" Expr ")" ] "{" Fields "}"
TraitDecl  ::= [ "pub" ] "trait" identifier [ GenericParams ] "{"
               TraitItems "}"
```

```
ImplDecl   ::= "impl" [ GenericParams ] Type [ "for" Type ] "{"
               ImplItems "}"

ContractDecl ::= "contract" identifier "{" { ContractItem } "}"
ContractItem ::= StateDecl
               | ErrorDecl
               | StructDecl | EnumDecl | MessageDecl
               | EntryFunctionDecl
               | GetterDecl
               | FunctionDecl

StateDecl ::= "state" "{" Fields "}"
ErrorDecl ::= "error" ErrorVariant { "," ErrorVariant } ";"
ErrorVariant ::= identifier "=" integer-literal

EntryFunctionDecl ::= EntryKind "fn" identifier "(" Params ")"
                      [ "->" Type ] [ EffectQualifier ] Block
EntryKind    ::= "external" | "internal" | "bounce"
GetterDecl   ::= "get" "fn" identifier "(" Params ")" "->" Type
                 [ MethodIdClause ] Block
MethodIdClause ::= "method_id" "(" integer-literal ")"
                 | "method_id" "(" string-literal ")"

FunctionDecl ::= [ "pub" ] "fn" identifier
                 [ GenericParams ] "(" Params ")" "->" Type
                 [ EffectQualifier ] BlockOrAsmBody

EffectQualifier ::= "pure"
                  | "unsafe"
                  | "effect" "(" EffectList ")"
                  | "unsafe" "asm" ( string-literal | "{" AsmBody "}" )

BlockOrAsmBody ::= Block
                 | ";"
                 | "asm" ( string-literal | "{" AsmBody "}" ) ";"

AsmBody ::= triple-quoted-string
```

```
InlineAsmExpr ::= "asm" "(" InVars ")" "->" "(" OutTys ")"
                  "{" AsmBody "}"

Stmt ::= LetStmt
       | ExprStmt
       | ReturnStmt
       | IfStmt | IfLetStmt
       | MatchStmt
       | WhileStmt | WhileLetStmt
       | LoopStmt | ForStmt
       | BreakStmt | ContinueStmt
       | RequireStmt
       | EmitStmt
       | Block

RequireStmt ::= "require!" "(" Expr "," Expr ")" ";"
EmitStmt    ::= "emit" Path "{" FieldInits "}" ";"

Type ::= SimpleType
       | Type "?"

Arg ::= Expr
      | "mut" Expr
```

Note that the grammar does **not** include a `?` postfix expression operator. A `T?` type is sugar for `Option<T>`; there is no expression-level try-operator.

19 Illustrative Contract Fragment

The following fragment demonstrates the overall style target — the full simplified surface with a `contract` block, named errors, typed entry parameters, and `require!`:

```
#![edition = "0.5.0"]

contract Wallet {
    state { seqno: u32, owner: Address }
```

```
error WrongSeqno = 33,
      ZeroAmount = 34;

message Transfer(op = 0x0f8a7ea5) {
    query_id: u64,
    amount:   Coins,
    to:       Address,
    response_to: Address?,
}

message Receipt(op = 0x00000001) {
    query_id: u64,
}

internal fn on_transfer(msg: Transfer) {
    require!(msg.amount > Coins::zero(), ZeroAmount);

    if let Some(reply) = msg.response_to {
        send(
            reply,
            Receipt { query_id: msg.query_id },
            SendOptions {
                value:       Coins::zero(),
                bounce:      BounceMode::NoBounce,
                body_layout: BodyLayout::Auto,
                mode:        SendMode::PayFeesSeparately,
            },
        );
    }

    send(
        msg.to,
        (),
        SendOptions {
            value:       msg.amount,
            bounce:      BounceMode::Body256,
            body_layout: BodyLayout::Auto,
            mode:        SendMode::PayFeesSeparately,
```

```
            },
        );

        state.seqno += 1;
    }

    get fn seqno() -> u32 { state.seqno }
}
```

20 Conclusion

FunC `0.5.0` becomes easier to read and write not by hiding TVM/TOS but by modeling it honestly through a compact high-level surface and a single `asm` escape hatch:

- a `contract` block that collapses state, errors, messages, entries, and getters into one declarative shape;
- typed messages as entry parameters, with the wrapper handling opcode check and field decoding;
- `Signed<T>` for the ubiquitous wallet-style signed-body pattern;
- trailing `Slice` / `Cell` fields to model "fixed header plus opaque tail";
- abort-by-default with named error codes via `require!`, preserving legacy exit-code ABI;
- `Option<T>`, exhaustive `match`, and `if let` for the cases that genuinely benefit from typed absence;
- `emit Event \{ ... \}` as the canonical form for log-style outbound messages;
- effects inferred by default, displayed through the ABI manifest and enforced by the keyword ceiling on entries;
- `asm \{ ... \}` as the single, well-defined escape hatch to raw Fift when the high level is not enough;

- trait- and module-based reuse for stdlib and library authors, kept out of the ordinary contract author's daily path.

The combination preserves the strengths of FunC — closeness to TVM, direct control over serialization and messaging, and efficient compilation — while making the source language dramatically easier to teach, review, and scale across a real contract ecosystem.

21 Canonical Decisions

The items below were open questions in an earlier draft. Each has now been fixed from first principles: minimum runtime cost, determinism across toolchains, fidelity to TVM/TOS, and alignment with existing TON conventions. Compilers claiming specification conformance must obey these rules exactly.

21.1 Canonical Decision 1: Stack Representation of Sum Types

An `enum`, `Option<T>`, or `Result<T, E>` value occupies exactly two TVM stack values:

```
(tag : int,  payload : tuple)
```

- `tag` is an ordinary TVM 257-bit signed integer. No bit packing is performed on the stack; that is a wire-format concern only.
- `payload` is an ordinary TVM tuple. Variant fields appear in declaration order. A zero-field variant uses the empty tuple `[]`. A one-field variant uses the singleton tuple `[v]`. Multi-field variants use tuples of the appropriate arity.

Tag values:

- `Option<T>`: `None = 0`, `Some = 1`.
- `Result<T, E>`: `Ok = 0`, `Err = 1`.

- User-defined enums without an explicit discriminant type: tags are assigned `0, 1, 2, ...` in declaration order.
- User-defined enums with an explicit discriminant type (`enum E : uN`) or explicit per-variant values: the declared numeric values are the stack tags.

Optimisers may locally unbox the payload when the unboxing is not observable at function, trait, or ABI boundaries, but the canonical model above is what every compiler must be able to produce on demand (for example at `store_any` boundaries).

21.2 Canonical Decision 2: Wire Format of Sum Types

The codec for a sum type S with N variants writes:

1. a tag prefix of width

$$w_S \;=\; \max(1,\ \lceil \log_2 N \rceil)$$

   bits, big-endian, unsigned, unless the enum declares an explicit discriminant type `: uN` in which case the tag is exactly `uN` wide;

2. the variant fields, in declaration order, each by its own codec.

For `Option<T>` the tag is 1 bit (`None = 0`, `Some = 1`), yielding a wire format bit-compatible with TL-B `Maybe X`. For `Result<T, E>` the tag is 1 bit (`Ok = 0`, `Err = 1`).

A `message M(op = X)` declaration is a special case: the wire prefix is the 32-bit opcode `X`, not a derived enum tag. A sum of `message` variants (for dispatch) uses the 32-bit opcodes of each constituent message as its discriminant; the enum does not insert an additional tag.

Adding, removing, or reordering variants of an enum *breaks the wire format*. Implementations must reject silent breakage at compile time by computing a schema hash over the enum definition and including it in the generated ABI manifest.

21.3 Canonical Decision 3: Layout of `Bytes`

`Bytes` is a stdlib struct with a fixed ABI:

- **Stack:** a single TVM `Slice` that is guaranteed byte-aligned (`remaining_bits() % 8 == 0`) when entering any function that accepts `Bytes`.
- **Wire:** the "snake" cell-chain convention already standard in TON: up to $\lfloor (1023 - \textit{header bits})/8 \rfloor$ bytes inline in the current cell, overflow spilled into a single tail reference whose referent is itself a snake cell. A length prefix is *not* part of the snake convention; the reader stops at end-of-slice.
- For protocols that need a length-prefixed byte string, stdlib provides `LenPrefixedBytes` with an explicit `u16` length.

`Bytes` is deliberately not a `Cell<Bytes>` because the most common pattern (message bodies, metadata strings) inlines the prefix into the current cell rather than forcing a ref.

21.4 Canonical Decision 4: `BodyLayout::Auto` Fit Predicate

Let `env` be the current envelope builder after the compiler has stored the common message header (source, destination, value, flags, created-lt, created-at, init, and the outer `Either` tag for the body). Let `body` be the encoded body after materialising it into a temporary builder. Define:

```
env_bits   = env.bits_used()
env_refs   = env.refs_used()
body_bits  = body.bits_used()
body_refs  = body.refs_used()
```

`BodyLayout::Auto` emits the *inline* form (`body:(Either X \^{}X) = Left X`) iff both of:

```
env_bits + 1 + body_bits <= 1023
env_refs +     body_refs <=    4
```

Otherwise it emits the *by-ref* form (`Right \^{}X`). The constant `+1` accounts for the outer `Either` tag bit. All compilers must compute exactly this predicate.

When `body_bits` is statically known (constant-size codec), the compiler may fold the branch at compile time. When it is not, the compiler emits a runtime branch whose test matches the predicate above bit-for-bit.

`BodyLayout::Inline` and `BodyLayout::ByRef` force the respective choice; a body that does not fit inline under `Inline` is a runtime error mapped to `CellError::Overflow`.

21.5 Canonical Decision 5: Abort Semantics and Exit Codes

The primary failure mechanism in FunC `0.5.0` is abort: a failed operation throws a numeric exit code, the transaction rolls back, no state or actions are committed. The mechanism is reached from source code via `require!(cond, err)` (§11.4) or by any stdlib function that itself aborts on underlying failure.

Exit-code derivation from error names. A `require!(cond, err)` where `err` is an `enum E : u16` variant throws exactly that variant's numeric discriminant. The `error` shorthand (§6.5) and the longer `enum E : u16 { ... }` form both expand to the same representation.

Reserved ranges:

- `0`: normal success. Assigning `0` to an error variant is a compile-time error.
- `1 – 10`: TVM-internal exit codes (8 = range check, 9 = cell underflow, 10 = dict error). Authors may re-use these codes when the semantics match.
- `11 – 31`: TOS-reserved for protocol-level failures such as unknown operation (11) and insufficient fee.
- `32 – 65535`: user range. Existing TON wallet and Jetton conventions occupy `32 – 400`.

Commit boundary. The wrapper around each entry function handles commit boundary uniformly: on normal return the wrapper commits state (saving the final value of `state`) and action-phase effects, then returns `0` as the exit code. On any `THROW` during the user body, the wrapper's own handler discards the action list, skips the `set_data` commit, and propagates the thrown exit code to TVM.

Rare `Result<T, E>` use case. If the user body explicitly returns `Err(e)` as a normal value (no `THROW`, the function's declared return type is `Result<T, E>`), the wrapper treats it as a successful return with a typed error payload. Whether that payload is visible to the outside world is the author's responsibility — typically by `emit`-ing a log message or `send`-ing a reply before returning. This pattern is rare; see §13.

21.6 Canonical Decision 6: Getter / `method_id` Selector Rule

For every `get fn name(...)`:

- the default exported method id is `crc16(name) \ 0x10000` (identical to FunC `0.4.6`, preserving interop with every existing TON/TOS indexer and wallet);
- an explicit override uses the same syntax family as `0.4.6`: `get fn name(...) method_id(N)` for a literal integer, or `method_id("string")` to select via the crc16 of an alternative name;
- collisions in the final exported method-id set are a compile-time error; the compiler reports every colliding pair;
- two sibling compilations that produce different method-id assignments for the same source are a violation of this rule.

The grammar for `FunctionDecl` is extended accordingly:

```
FunctionDecl ::= [ "pub" ] [ EntryQualifier ] "fn" identifier
                 [ GenericParams ] "(" Params ")" "->" Type
                 [ EffectQualifier ]
                 [ "method_id" ( "(" IntLiteral ")"
```

```
                                | "(" StringLiteral ")" ) ]
                BlockOrSemicolon
```

`method_id` is admissible only on `get fn` functions in edition `0.5.0`; using it on a non-getter is a compile-time error.

21.7 Canonical Decision 7: Expression Grammar and Precedence

The expression grammar below supersedes the earlier schematic summary. It is parsed top-down with precedence climbing.

```
Expr        ::= Assign
Assign      ::= Or [ AssignOp Assign ]            // right-assoc
AssignOp    ::= "=" | "+=" | "-=" | "*=" | "/=" | "%="
              | "&=" | "|=" | "^=" | "<<=" | ">>="

Or          ::= And { "||" And }                  // left-assoc, short-circuit
And         ::= Compare { "&&" Compare }          // left-assoc, short-circuit
Compare     ::= BitOr [ CmpOp BitOr ]             // non-associative
CmpOp       ::= "==" | "!=" | "<" | ">" | "<=" | ">=" | "<=>"

BitOr       ::= BitXor { "|" BitXor }             // left-assoc
BitXor      ::= BitAnd { "^" BitAnd }             // left-assoc
BitAnd      ::= Shift  { "&" Shift }              // left-assoc
Shift       ::= Add    { ( "<<" | ">>" ) Add }    // left-assoc
Add         ::= Mul    { ( "+" | "-" ) Mul }      // left-assoc
Mul         ::= Unary  { ( "*" | "/" | "%" ) Unary }

Unary       ::= { UnaryOp } Cast
UnaryOp     ::= "-" | "!" | "~"
Cast        ::= Postfix [ "as" Type ]

Postfix     ::= Primary { PostfixOp }
PostfixOp   ::= "." Ident [ "(" Args ")" ]        // field or method call
              | "(" Args ")"                      // function call
              | "[" Expr "]"                      // index
              | "::" Ident                        // associated fn/const path
```

```
Primary    ::= Literal
             | "(" ")" | "(" Expr { "," Expr } ")"
             | "[" "]" | "[" Expr { "," Expr } "]"
             | StructLit
             | EnumLit
             | Path
             | "if" Expr Block [ "else" Block ]
             | "match" Expr "{" { MatchArm } "}"
             | Block

Args       ::= [ Arg { "," Arg } ]
Arg        ::= Expr | "mut" Expr
```

Precedence summary (highest to lowest):

Level	Forms	Associativity
14	postfix: `.`, `(..)`, `[..]`, `::`	left
13	prefix: `-`, `!`, `~`; `as`	right
12	`* / %`	left
11	`+ -`	left
10	`<< >>`	left
9	`&`	left
8	`^`	left
7	`|`	left
6	`== != < > <= >= <=>`	non-associative
5	`&&`	left (short-circuit)
4	`||`	left (short-circuit)
2	`=` and compound assignments	right

Notes:

- Comparisons are non-chainable: `a < b < c` is a syntax error.
- FunC `0.5.0` has no `?` postfix operator; the error model is abort-by-default via `require!` (§11.4).
- `as` takes a type, not an expression, on the right.

- Bitwise `&`/`|`/`^` coexist with logical `&&`/`||` at separate precedences. Logical operators require `bool` operands and short-circuit; bitwise operators work on integer types only.

21.8 Canonical Decision 8: Trait Coherence

FunC `0.5.0` adopts an orphan-style coherence rule:

- `impl Trait for T` is admissible iff `Trait` *or* `T` is declared in the current compilation unit (the file or the set of files forming one `mod`).
- A blanket `impl<X: Bound> Trait for X` is admissible iff `Trait` is declared in the current compilation unit.
- For any concrete pair `(Trait, T)` there may be at most one `impl` in the full dependency graph; a second impl is a compile-time error. This includes indirect duplication through blanket impls.
- The earlier `impl<T: CellEncode> T { fn to_cell(...) }` shorthand in §7 is not admissible. `to_cell` is instead a method on the `CellEncode` trait itself, either abstract or as a default method using the trait's `encode`:

```
pub trait CellEncode {
    fn encode(self) -> RawCell pure;
    fn to_cell(self) -> Cell<Self> pure {
        Cell::from_raw(self.encode())
    }
}
```

Coherence is checked after monomorphisation: an instantiated concrete pair must not acquire two impls via different routes.

21.9 Canonical Decision 9: Entry Parameter as Typed Message

Every entry function inside a `contract { }` block takes its inbound body directly as a typed parameter:

```
external fn recv(msg: Signed<Payload>)   { ... }
internal fn recv(msg: WalletMsg)          { ... }
bounce   fn on_bounce(ctx: BounceContext<Transfer>) { ... }
```

The generated wrapper decodes the raw body into the declared parameter type using the derived codec, then invokes the user body. This replaces the earlier v3 design where entry functions took a context wrapper object (`InternalContext<T>`, `ExternalContext<T>`, `BounceContext<T>`) and needed either "eager" or "lazy" decoding markers.

Decode-failure semantics are specified in §10.6:

- `internal fn`: decode failure → silent drop;
- `external fn`: decode failure → abort with exit code 9 (cell underflow). The wrapper's decode runs *before* any user code, so it must be cheap enough to complete under the pre-`accept\_message` gas quota — which the typical "fixed header + trailing `Slice`" pattern satisfies;
- `bounce fn`: decode uses the bounced-message convention subject to the 256-bit / zero-ref constraint (§10.6).

Using a plain `Slice` parameter. An entry may also take a plain `Slice`; the wrapper performs no decoding and the user body is responsible for all parsing. This is the low-ceremony escape when the typed-message pattern is not a good fit (for example, contracts that want to inspect raw bytes before deciding how to parse).

Ceiling context types. The stdlib provides `BounceContext<T>` because the bounce protocol carries metadata beyond the body; `Signed<T>` because wallet-style signed bodies have a standard wrapper shape. These are *not* compiler-magic wrappers; they are ordinary user-visible `message` declarations in the standard library.

21.10 Canonical Decision 10: Compatibility Edition Surface

An edition is chosen per source file by an inner attribute; it is not per-item.

- `#![edition = "0.5.0"]` selects the strict `0.5.0` surface. This is the recommended edition for new code.
- `#![edition = "0.4-compat"]` selects the superset edition that accepts all `0.4.6` constructs plus all `0.5.0` constructs. The declared purpose is migration, not long-term authoring.
- Absence of an edition attribute defaults to `0.4-compat` for this release, so existing contracts continue to compile without modification.

Under `0.4-compat` the following legacy constructs are accepted in addition to the `0.5.0` surface:

- lexical: `;;` line comments; `{- ... -}` nested block comments; legacy string suffixes `s`, `a`, `u`, `h`, `H`, `c`; `?`-suffixed predicate identifiers; `~`-prefixed modifying methods;
- top-level: `#pragma ...`, `#include ...`, `global` declarations, `const` declarations without type, function bodies introduced by `asm` with `method_id` after modifiers;
- function modifiers: `impure`, `inline`, `inline_ref`, `method_id` in `0.4.6` positions;
- types: `int` as an alias for `i257`, and lowercase `cell`, `slice`, `builder`, `cont`, `tuple` as aliases for the `0.5.0` domain types;
- statements: `repeat Expr Block`, `do Block until Expr`;
- expressions: `var ...` bindings, explicit type application `(int, slice) (n, s) = ...` on the left of `=`, bare unit return `return ()`; method calls of the form `receiver~name(args)`.

Cross-edition interoperation:

- a `0.5.0` file may `use` items from a `0.4-compat` file, provided the imported items have types expressible in the `0.5.0` surface;
- a `0.4-compat` file may call `0.5.0` items that do not use features absent from the compat surface (notably: `Result`, typed `entry`, traits, or enums);

- a `0.4-compat` file may *not* define a `0.5.0` `entry` function; entry points cross the compatibility boundary exactly once, at the outer edition of the contract's root module;
- the contract's root module's edition determines the emitted ABI.

Removal schedule. `0.4-compat` is a migration surface, not a long-term supported edition. It is scheduled for removal in a future edition `0.7.0`. Contracts that have not migrated by then must freeze on the last compiler release that still accepts it.

22 Canonical Corpus

The 10 Canonical Decisions above are normative on paper; the corpus is how they are validated against real contracts and how cross-compiler conformance is checked. The corpus is defined in a sibling document, `doc/func_v0.5.0-corpus.md`, which ships as version-pinned artefacts under `crypto/smartcont/v050/`.

Every compliance-seeking compiler must produce byte-identical TVM code-cell output for every corpus entry at its pinned version. Corpus v1 includes:

- C1 — Simple wallet (`wallet3`);
- C2 — Plugin wallet (`wallet-v4`);
- C3 — Highload wallet v2;
- C4 — Jetton wallet (synthetic, from TEP-74);
- C5 — Multisig (sketch; full rewrite in corpus v2);
- — Elector (deferred to corpus v2).

The corpus is intentionally small in v1: enough to pin the language, not so large that freezing it blocks edition maturation. Corpus versions are monotonic — v2 may add entries but may not redefine entries already fixed in v1, short of a spec-revision-grade compatibility break.

Issues surfaced by corpus v1, now resolved in main spec v3. Every issue that corpus v1 flagged as open has been folded into the main specification:

- helper-function state mutation — resolved in §9.6 by the existing `mut` parameter qualifier; there is no `\&mut self` syntax;
- `Vec<T>` at getter and ABI boundaries — resolved in §7.3;
- `EitherRefOrInline<T>` per-field encoding — resolved in §7.4;
- bounce-body width enforcement — resolved in §10.6 by the `BounceBodyTooWide` compile-time check;
- canonical `std::` module layout — resolved in Appendix A below.

A Standard Library Module Map

The canonical `std::` module tree below is part of the edition-`0.5.0` ABI. Every conforming compiler must accept these module paths and export the listed names. Item signatures are authoritative in the modules themselves (shipped alongside the reference compiler); this appendix fixes the *tree* so that `use` paths across source bases are compatible.

Module path	Key exports
`std::prelude`	auto-imported into every source file: `Option`, `Some`, `None`; `Result`, `Ok`, `Err` (rare use); `require!` macro form; `emit` macro form
`std::cell`	`Cell<T>`, `RawCell`, `Builder`, `Slice`, `RawSlice`; conversions `to_cell`, `from_cell`, `load_any`, `store_any`; `Cell::from_raw`, `Cell::as_raw`
`std::address`	`Address`, `StdAddress`, `parse_std_addr`, `Address::derive`, address codecs
`std::coins`	`Coins`, `Coins::zero`, `Coins::is_zero`, arithmetic, safe conversions
`std::bytes`	`Bytes`, `LenPrefixedBytes`, helpers for snake encoding and byte-alignment assertions

Module path	Key exports
`std::vec`	`Vec<T>`, `EitherRefOrInline<T>`; iteration traits on `Vec`
`std::dict`	`Dict<K, V>`, `DictEntry<K, V>`, iteration via `iter_sorted`, `Dict::pop_min`, `Dict::contains`, `Dict::insert`, `Dict::remove`, `Dict::get`
`std::crypto`	`check_signature`, `slice_hash`, `sha256`, `keccak256`, curve primitives that expose TVM hashing opcodes
`std::chain`	`now`, `global_id`, `accept_message`, `raw_reserve`, `get_data`, `set_data`, contract-balance and block-context readers
`std::message`	`send`, `send_raw`, `SendOptions`, `SendMode`, `BounceMode`, `BodyLayout`; `Signed<T>`, `BounceContext<T>`, `SendError`; log-sink address constant and `emit` sink resolution
`std::iter`	`Iter`, `IntoIter` traits (see §21.7); stdlib iterator impls for `Vec`, `Dict`, ranges, fixed arrays, uniform tuples
`std::option`	combinators on `Option`: `map`, `unwrap`, `unwrap_or`, `is_some`, `is_none`, `ok_or`
`std::result`	combinators on `Result`: `map`, `map_err`, `and_then`, `or_else`, `ok_or`. Reminder: idiomatic 0.5.0 code uses `require!` instead of `Result` pipelines
`std::intrinsics`	low-level `extern "tvm"` declarations for every TVM opcode that the stdlib surface wraps, available to `unsafe` code only

Module-path collision rules:

- third-party libraries may not publish into the `std::*` namespace;
- contract code may publish its own `mod foo::bar` tree freely;
- the `use` resolver searches the current crate first, then `std::*` as a reserved root, then external dependencies listed in the build manifest;

- `use std::prelude::*;` is applied implicitly at the top of every source file under edition `0.5.0`.

A full item-level stdlib reference (every exported signature with documentation) is scheduled as a sibling document, `doc/func_v0.5.0-stdlib-ref.md`, to be written alongside the reference compiler implementation. The tree above is the frozen surface; the signatures inside each module may still evolve up to edition freeze.